



SAP Commerce Cloud - Code Assessment mtel.me - B2C

2020-12

LEGAL DISCLAIMER

The contents of this document shall remain the confidential property of SAP and may not be communicated to any other party without the prior written approval of SAP. This document must not be reproduced in whole or in part. It must not be used other than for evaluation purposes only by Customer except with the prior written consent of SAP and then only on condition that SAP's and any other copyright notices are included in such reproduction.

The furnishing of this document is subject to contract and shall not be construed as an offer or as constituting a binding agreement on the part of SAP to enter into any relationship. SAP provides this document as guidance only to estimate costs and timescales of the predicted delivery project. This will be subject to confirmation prior to any contractual or delivery commitment by SAP. Any services to be provided by SAP as proposed in this document shall be subject to the standard terms and conditions of SAP for the provision of the relevant services.

SAP has no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, referenced documents and SAP's strategy and possible future developments, products and /or platform directions and functionality are all subject to change and may be changed from SAP at any time for any reason and without notice. The information in this document and any referenced documents is provided without a warranty of any kind, either express or implied, including but not limited to, the implied warranties of merchantability, fitness for a particular purpose, or non-infringement. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Customer is cautioned not to place undue reliance on these forward-looking statements, and they should not be relied upon in making purchasing decisions.

Those who prepared this response have taken reasonable care in preparing it and believe its contents to be true as at the date of this document. SAP cannot warrant the truth of matters outside of its control and accordingly does not warrant the truth of all statements set out in this document to extent that such statements derive from facts and matters supplied by other persons to SAP. The statements in this document are qualified accordingly.

DOCUMENT REVISION HISTORY

Date	Revision	Author	Summary of Changes
11/12/2020	1.0	franciszek.bieg@sap.com	Final version

TABLE OF CONTENTS

01 EXECUTIVE SUMMARY	7
02 INTRODUCTION	8
Introduction	8
Recommendations	8
03 PROJECT SETUP	11
Overview	11
Extension Configuration.....	12
IDE Configuration.....	13
Source Control	13
ImpEx Files	14
Automated Deployment	15
The Manifest file	16
04 SOURCE CODE ANALYSIS	20
Overview	20
SonarQube	20
Evaluation Scope	20
Sonar Results	21
CPD Analysis	23
Complexity Report	23
FindBugs and PMD	25
Class Conflicts	27

05 DATA MODEL	28
Overview	28
Types	28
Type System Validation	28
06 VISUAL CODE ANALYSIS.....	33
Overview	33
General Inspection	33
Accelerator/Commerce Extensions	38
Usage of Logging	38
Unit Tests	39
Controllers	40
Facades	40
Services.....	40
DAOs	40
Populators and Converters	40
Value Providers	41
Interceptors.....	41
Spring	42
Session Management.....	43
Usage of Services provided by SAP Commerce Solution.....	43
Cache.....	44
07 BACKOFFICE CONFIGURATION	45
Overview	45
Deprecated Features	45
Backoffice - Application Configuration	45
Custom Backoffice Behavior	46

Backoffice Folder Structure 46

Customer Specific Data Model..... 47

Localization..... 47

01 EXECUTIVE SUMMARY

This document describes the results of the Code Assessment conducted by the SAP team for the mtel.me - B2C in December 2020.

This review identified that project is generally respecting the SAP Commerce best practices, however the following observations are considered of importance:

1. There is one high priority issues and three medium priority related to code quality.
2. Logging integration requests to database can cause performance issues. Consider using standard file logging which has log monitoring infrastructure needed for analysis.
3. Review and fix data model recommendations.
4. Implement configurable timeouts for integration interfaces.
5. Application uses default API and region cache configuration. Fine tune cache configuration based on the performance tests executed before Go-Live.

It is suggested that the above is addressed during further project implementation.
For the full list of findings and their description please refer to the rest of the document.

02 INTRODUCTION

Introduction

The Code Assessment is typically delivered iteratively during the engineering phase of the project to ensure that any quality and efficiency issues are addressed early on and often. The review validates that accepted industry and SAP Commerce Cloud Solution best practices are followed to minimize the risk of performance issues and that the code base will not be difficult to maintain.

Without an independent review, there is a risk that the delivered solution is not of high quality and doesn't follow best practices for SAP Commerce Cloud or SAP Commerce Solution projects. This could introduce performance and scalability issues and make the solution more difficult to maintain and upgrade to newer releases.

For this review, we used the version from **1 December 2020 (commit feb4532)** of the source code.

Activities

The review covers static-code analysis, data model and type checking, and a manual review of the source code. In each section we provide a list of recommendations for the issues identified.

This review is aimed at validating the custom-source code developed for the project and ensuring that it meets the SAP Commerce Solution implementation best practices and patterns. We also assess the appropriate usage of core libraries to ensure maintainability and upgradeability of the customizations. Please note that this is a source-code review and not a full-solution-architecture review. This review focuses on static-analysis tools and visual analysis of the source code provided.

- The **Data Model** is analysed to ensure that SAP Commerce best practices have been followed.
- All **Source Code** is manually reviewed for best use of APIs, and for general Java best practices.
- **Backoffice Configuration** is reviewed to ensure that tools will be usable for business users.
- **Review of Automated Testing** practices, as testing reduces the risk of defects being introduced.
- The **Configuration** of the project is reviewed to ensure the correct use of modules, project data files and dependencies between extensions.

Recommendations

Findings and recommendations that are part of this review are detailed within each subsection of this document. In this section, we summarize all recommendations and provide a *priority* and estimated *level of effort*. This is intended to allow the team to weigh each recommendation to determine which items should be prioritized in future development cycles.

Conventions

Each of our recommendations has a:

1. **Unique key**: the key's prefix indicates the area of the recommendation.
2. **Short description**
3. **Priority**: LOW, MED, or HIGH.

The priority is indicated by the recommendation's coloured box.

Following SAP Commerce Cloud best practices:

This recognizes items that are following best practices. No action required.

Low priority recommendation:

This recommendation should be reviewed and worked into a longer-term plan, as it may have impact on future growth and performance.

Medium priority recommendation:

This recommendation should be reviewed and factored into upcoming releases as feasible.

High priority recommendation:

This recommendation should be reviewed and designated as a priority in the short term.

Level of Effort estimate: LOW, MED, or HIGH.

The following table is a rough estimate to how much time is required to correct each Level of Effort rating. Note that this is just an estimate and not an official guide for how long these items will take.

Level of Effort	Description
HIGH	High effort items will typically take more than a week.
MED	Medium effort items can typically be resolved in a week or less.
LOW	Low effort items can typically be resolved in a few days or less.

List of Recommendations

The following tables summarize the recommendations and is intended to assist in project prioritization. The level of effort is an estimate and does not take in to account the experience level of the project team.

Project Setup

Key	Recommendation	Priority	Level of Effort
PS-01	Consider integrating SAP Commerce Cloud with CI process by using Build and Deployment APIs: https://help.sap.com/viewer/0fa6bcf4736c46f78c248512391eb467/v2011/en-US/a1db2e0af60247189783184c94ca3773.html	LOW	MED

Source Code Analysis

Key	Recommendation	Priority	Level of Effort
SC-01	Expressions should not be too complex: CartsController:187	MED	LOW
SC-02	Control flow statements "if", "for", "while", "switch" and "try" should not be nested too deeply: MtmAuthenticationSuccessCustomerFlowExecutorEventListener:58, MtmMobileResourcesInfoPopulator:64	MED	LOW
SC-03	When handling a caught exception, the original exception's message and stack trace should be logged or passed forward: MtmCustomerFacadeImpl:262,	HIGH	LOW
SC-04	Review and fix Major code issues.	MED	MED

Data Model

Key	Recommendation	Priority	Level of Effort
DM-01	For coding best practices, attribute names should start with an lowercase letter: Customer.JMBG	MED	LOW
DM-02	Any side of a relation that has cardinality='many' should not have ordered='true' unless absolutely necessary.	LOW	LOW
DM-03	No database indexes defined for unique attributes of type: MtmRegistration, MtmOtpRequest, MtmDeviceInfo.	MED	LOW
DM-04	Logging integration requests to database can cause database overload which can affect storefront performance. Consider using standard log file which is searchable by Kibana.	HIGH	MED

Visual Code Analysis

Key	Recommendation	Priority	Level of Effort
VCA-01	Application uses default API and region cache configuration. Fine tune cache configuration based on the performance tests executed before Go-Live.	LOW	MED
VCA-02	Implement configurable timeouts for integration interfaces.	MED	LOW
VCA-03	Remove V1 version REST API if not used.	LOW	LOW

03 PROJECT SETUP

- [Overview](#)
- [Extension Configuration](#)
- [IDE Configuration](#)
- [Source Control](#)
- [ImpEx Files](#)
 - [Dynamic Attributes](#)
- [Automated Deployment](#)
- [The Manifest file](#)

Overview

This section defines areas that relate to the overall project and its setup. These items are important to the overall health and success of a project even though they do not deliver customer-facing functionality.

Extension Configuration

Here is the current setup found in **localextensions.xml** file.

```
<path dir='${HYBRIS_BIN_DIR}' autoload='false' />

<extension name='b2ctelcobackoffice' />

<extension name='b2ctelcowebservices' />

<extension name='b2ctelcoservices' />

<extension name="adaptivesearchbackoffice" />

<extension name="adaptivesearchsolr" />

<extension name='commerceservicesbackoffice' />

<extension name='solrserver' />

<extension name='subscriptionbackoffice' />

<extension name='cmswebservices' />

<extension name='customersupportbackoffice' />

<extension name='acceleratorsservices' />

<extension name='ldap' />

<extension name="patchesbackoffice"/>

<extension name="mtmcore"/>

<extension name="mtminitialdata"/>

<extension name="mtmfacades"/>

<extension name="mtmtest"/>

<extension name="mtmbackoffice"/>

<extension name="mtmcommercewebservices"/>

<extension name="mtmmockserver"/>

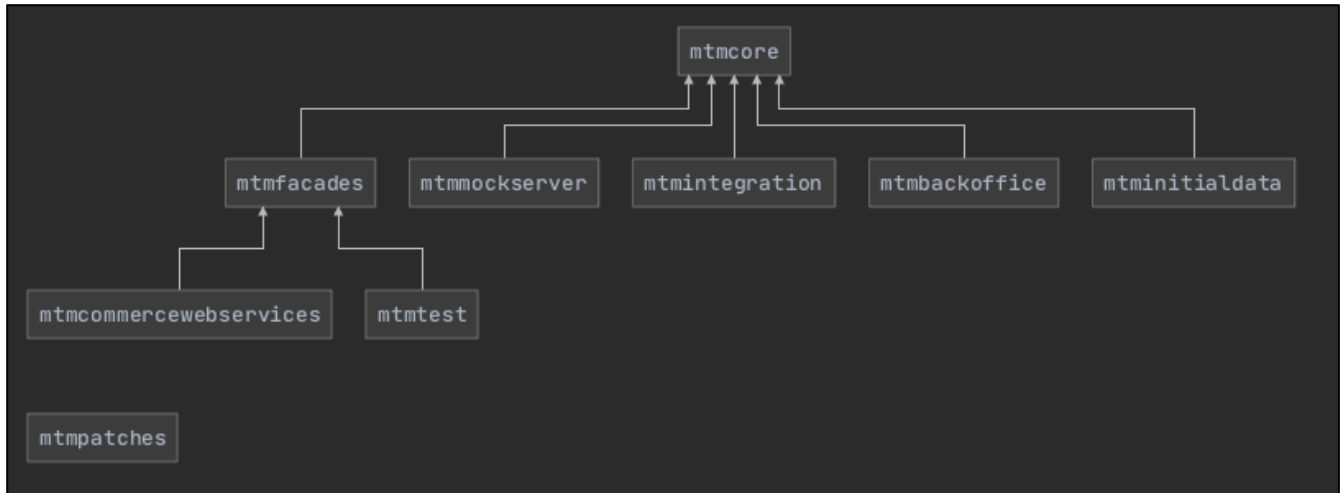
<extension name="mtmintegration"/>

<extension name="mtmpatches"/>
```

Extensions found in the solution are used and necessary.

SAP Commerce Solution recommends using different extensions for the layers of the application (web, facades, services), as well as for integrations with other systems in order to reinforce a separation of concerns.

It's important to ensure the dependencies are correctly configured for a project to ensure we can reuse functionality without introducing cyclic dependencies that might force us to copy and paste code between extensions or move code into the wrong layer of an application.



This project uses recommended dependency structure and does not include any cyclical dependencies.

IDE Configuration

It is important that the custom code base is configured so that the IDE being used by developers is working with minimal-local-configuration required. This means that the developer should be able to checkout a fresh copy of the source code, import the projects, and have a clean workspace that builds correctly. All project dependencies and classpath issues should be resolved and committed to the repository along with the code base.

IDE configuration guidelines were found and could be executed without issues.

Source Control

It is important that the source control system is correctly used for the project. SAP Commerce Solution generated many files that should not be committed to the repository and should be ignored by the source-control system.

In addition it is important to use a branch and tagging strategy to ensure branches can work in parallel with minimal impact on each other. This makes it easier to manage, maintain, and support both internal and external releases.

There are a number of concerns that should always be satisfied through the use of source control:

- The current project should make use of tags and branching. In a collaborative project using Git the recommendation is to use feature branches for development and tagging to track versions.
- Source code comments should be linked to a work-item reference (e.g. Jira ticket). This assists in terms of tracking why changes have been made and for what.
- Pull requests should always be created before merging new code. It is recommended to review, discuss, and approve code changes related to the pull requests before merging any feature branch to master.

The concerns above have all been satisfied by source-control system found in this project.

ImpEx Files

SAP Commerce Cloud Solution recommends that each project create the seed data for provisioning a new system in the form of ImpEx files. This allows new development to be setup very quickly. The distinction is made between essential data that is not expected to change between environments (Languages, Countries, Catalogs, etc.) and sample project data (Products, CMS Components, Media, etc.). Essential data and sample project data should be able to be loaded independently during a system update and it should be possible to disable the loading of sample project data entirely.

Creation of seed data has been properly provisioned.

Dynamic Attributes

Errors can occur during initialization if you try synchronize dynamic attributes. Example:

```
ERROR [SyncWorker<00000022 10 of 16>] (00000022) [accessor] No accessor found for
attribute 'sapNITransmissionDetail' of type
'nz.co.foodstuffspim.core.jalo.FSBaseProduct'. It seems that this attribute was removed
from items.xml. If this was intended please remove the related attribute descriptor
manually, for example using the hMC.
```

```
ERROR [SyncWorker<00000022 10 of 16>] (00000022) [accessor] No accessor found for
attribute 'sapSITransmissionDetail' of type
'nz.co.foodstuffspim.core.jalo.FSBaseProduct'. It seems that this attribute was removed
from items.xml. If this was intended please remove the related attribute descriptor
manually, for example using the hMC.
```

Such attributes can be excluded from synchronization rule through ImpEx as well:

```
INSERT_UPDATE
SyncAttributeDescriptorConfig;syncJob (code) [unique=true];attributeDescriptor (enclosingType (code), qualifier) [unique=true];includedInSync;copyByValue;

;sync FSProductCatalog\Staged-
>FSProductCatalog\Online;FSBaseProduct:sapNITransmissionDetail;false;false

INSERT_UPDATE
SyncAttributeDescriptorConfig;syncJob (code) [unique=true];attributeDescriptor (enclosingType (code), qualifier) [unique=true];includedInSync;copyByValue;

;sync FSProductCatalog\Staged-
>FSProductCatalog\Online;FSBaseProduct:sapSITransmissionDetail;false;false
```

No dynamic attributes are synchronized.

Automated Deployment

Continuous integration can be considered completed when an automated deployment process has been put in place. Three benefits can be expected from this point:

- Continuous process installation testing
- Quick and efficient deployment
- Complete understanding of each installation step and their dependencies

PS-01

Consider integrating SAP Commerce Cloud with CI process by using Build and Deployment APIs: <https://help.sap.com/viewer/0fa6bcf4736c46f78c248512391eb467/v2011/en-US/a1db2e0af60247189783184c94ca3773.html>

The Manifest file

Code Block 1 manifest.json


```

{
  "commerceSuiteVersion": "2005",
  "disableImageReuse": false,
  "extensionPacks": [
    {
      "name": "hybris-b2ctelco-suite",
      "version": "2007.0"
    }
  ],
  "extensions": [
    "backoffice"
  ],
  "useConfig": {
    "properties": [
      {
        "location": "build/config/common/local.properties"
      },
      {
        "location": "build/config/dev/local.properties",
        "persona": "development"
      },
      {
        "location": "build/config/stage/local.properties",
        "persona": "staging"
      },
      {
        "location": "build/config/prod/local.properties",
        "persona": "production"
      }
    ]
  },
  "extensions": {
    "location": "build/config/common/localextensions.xml",
    "exclude": [
      "mtmtest"
    ]
  },
  "solr": {
    "location": "core-customize/solr-conf"
  },
  "aspects": [
    {
      "name": "backoffice",
      "webapps": [
        {
          "name": "hac",
          "contextPath": "/hac"
        },
        {
          "name": "mediaweb",
          "contextPath": "/medias"
        },
        {
          "name": "backoffice",
          "contextPath": "/backoffice"
        }
      ]
    },
    {
      "name": "backgroundProcessing",
      "webapps": [
        {
          "name": "hac",
          "contextPath": "/hac"
        },
        {
          "name": "mediaweb",
          "contextPath": "/medias"
        }
      ]
    }
  ],
  {
    "name": "api",
    "properties": [
      {

```

```

        "key": "xss.filter.header.X-Frame-Options",
        "value": ""
    },
    "webapps": [
        {
            "name": "mtmcommercewebservices",
            "contextPath": "/mtmcommercewebservices"
        },
        {
            "name": "oauth2",
            "contextPath": "/authorizationserver"
        },
        {
            "name": "cmswebservices",
            "contextPath": "/cmswebservices"
        },
        {
            "name": "permissionswebservices",
            "contextPath": "/permissionswebservices"
        },
        {
            "name": "mediaweb",
            "contextPath": "/medias"
        },
        {
            "name": "mtmmockserver",
            "contextPath": "/mtmmockserver"
        },
        {
            "name": "hac",
            "contextPath": "/hac"
        }
    ],
    {
        "name": "accstorefront",
        "webapps": [
            {
                "name": "mediaweb",
                "contextPath": "/medias"
            }
        ]
    }
]
}

```

Property	Description	Value
commerceSuiteVersion	The exact version of the Commerce Suite artefact to download.	2005
extensions	The list of extensions to be enabled	There is one extension defined: backoffice
extensionPacks	The extension-packs section contains a list of named and versioned artifacts	hybris-b2ctelco-suite 2007.0
storefrontAddons	The AddOns installed by the Commerce Platform during the build process.	No addons
aspects	The aspect-specific configuration.	Configured aspects: backoffice, backgroundProcessing, api, accstorefront

useConfig	Allows the reuse of the configuration available in the repository with a customer project. It consists of extensions, properties, Solr configuration, and languages.	There is common config and per persona
tests	This section is used to configure extension tests to run during the build process	Not configured
webTests	This section is used to configure webTests to run during the build process	Not configured
properties	A list of common properties to be set in the local properties for all aspects (key-value pair)	There is one property on API aspect: xss.filter.header.X-Frame-Options=

This review identified that the manifest.json definition is correct and contains all of the expected configurations.

1. commerceSuiteVersion is configured to use the latest patch instead of explicitly defining a patch.
2. Configurations have been reused to make local development as close to what will be deployed in production.

04 SOURCE CODE ANALYSIS

Overview

This section is a static source code analysis of the custom Java codebase using SonarQube (www.sonarqube.org). The outcome of this analysis will be *quality measures* (e.g. code duplication, code complexity, unit test coverage, etc.) and *issues* (i.e. instances where coding rules were broken). The focus is on quality measures and high severity issues that have an impact on bugs, performance, security, and maintenance.

SonarQube

EC-hybris-Java Quality Profile

SonarQube scans the source code and checks if best practice *rules* are being followed. The list of *rules* (and their severities) are defined in a *quality profile*. SonarQube provides a default quality profile, but SAP has created its own SonarQube quality profile with a custom blend of rules to accurately measure the code quality of an SAP Commerce project. This quality profile is called **EC-hybris-Java** and is included with the SAP Commerce platform. It can be found at: **<SAPCommercePlatformDirectory>/build-tools/sonarqube/java-hybris-profile.xml**

Incorporating SonarQube into the Project

SonarQube was used for this code review and we recommend [mtel.me - B2C](#) also setup and incorporate SonarQube analysis into both its development and continuous integration (CI) processes so that the development team can proactively address code quality issues as they arise.

Incorporating SonarQube into the project includes:

- Setting up a SonarQube server.
- Configuring the SonarQube server to run the "EC-hybris-Java" quality profile and optionally adding custom rules on top of this quality profile.
- Configuring the continuous integration (CI) process to run SonarQube scans against the codebase on a regularly schedule (e.g. nightly).
- Creating development processes to regularly check, prioritize, and fix issues detected by SonarQube. We recommend leveraging SonarQube alerts.
- (Optional) Setting up the developers' IDEs so that the same SonarQube checks are done for each developer as they code. See <https://www.sonarlint.org/> for more information.

More information about setting up SonarQube for a SAP Commerce project can be found here: https://www.sap.com/cxworks/article/432587418/Measuring_Code_Quality_with_Sonar

Evaluation Scope

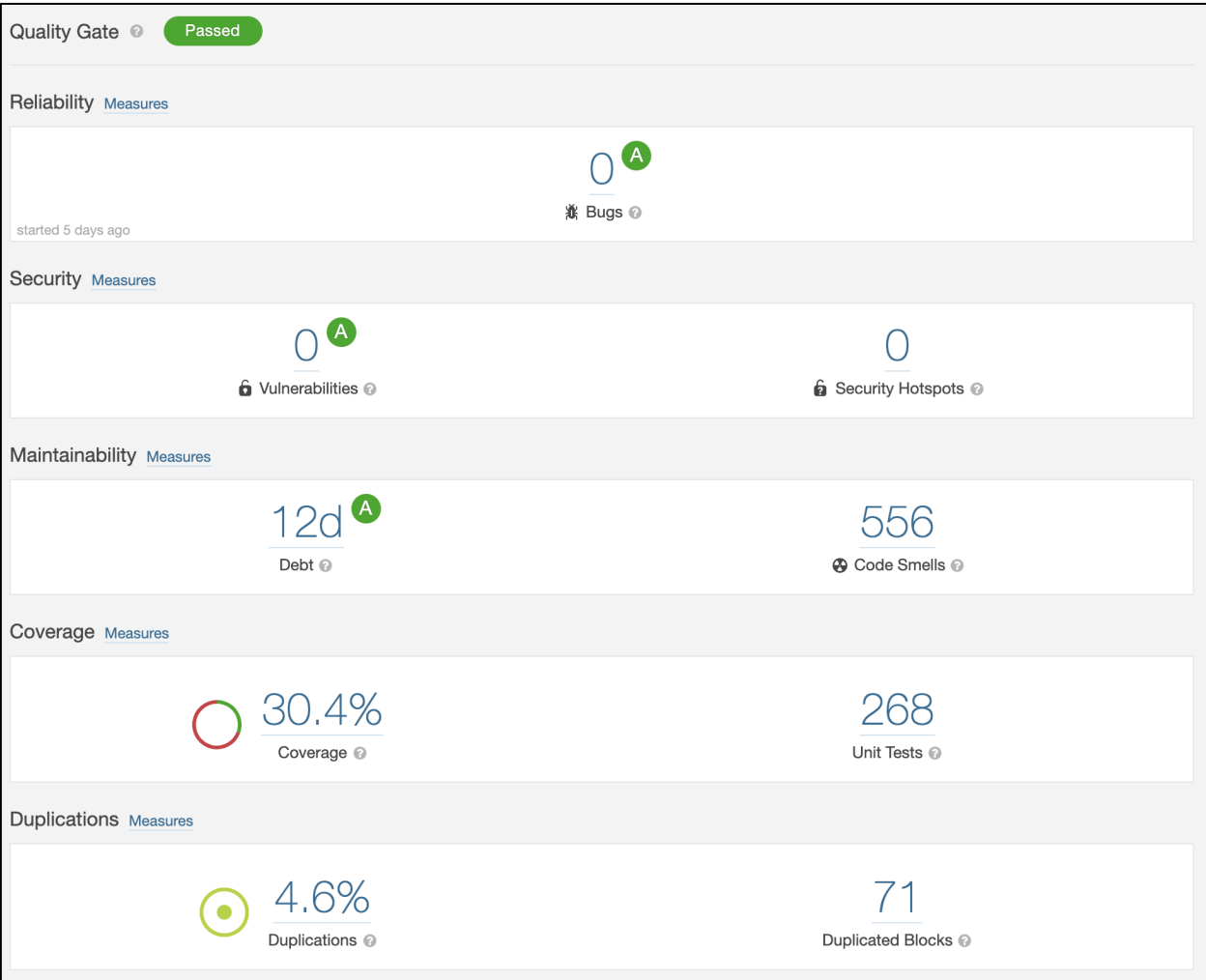
We have run SonarQube against the custom extensions implemented for this project and compared them with the results obtained from the SAP Commerce Cloud Accelerator (OOTB version) of same version.using the same configuration and rules. SonarQube was configured to run the (PMD, CPD, and FB) rule sets.

The following dashboard contains an overview of all components targeted during this evaluation.

	Lines of Code	Bugs	Vulnerabilities	Code Smells	Security Hotspots	Coverage	Duplications
hybris/bin/custom/mtm	32,010	0	0	556	0	30.4%	4.6%
mtmbackoffice	614	0	0	3	0	0.0%	0.0%
mtmcommercewebservices	15,030	0	0	287	0	4.2%	2.5%
mtmcore	6,980	0	0	122	0	53.7%	8.1%
mtmfacades	3,511	0	0	36	0	56.4%	9.7%
mtminitialdata	396	0	0	4	0	6.6%	0.0%
mtmintegration	3,737	0	0	50	0	69.3%	1.0%
mtmmockserver	818	0	0	4	0	0.0%	0.0%
mtmpatches	160	0	0	10	0	63.0%	11.7%
mtmtest	764	0	0	40	0	7.4%	5.2%

Sonar Results

The following dashboard contains an overview of the current status of the code.



We learned that a Sonar reviews are being made frequently, or before every release. Also, there is a proper "hardening" phase (or similar practice) to handle technical debt.

Reference for Sonar Metric Definitions

Extracted from: <https://docs.sonarqube.org/display/SONAR/Metric+Definitions>

Severity Types

Severity	Description
Blocker	Operational/security risk : This issue might make the whole application unstable in production. Ex: calling garbage collector, not closing a socket, etc.
Critical	Operational/security risk : This issue might lead to an unexpected behavior in production without impacting the integrity of the whole application. Ex: NullPointerException, badly caught exceptions, lack of unit tests, etc.
Major	This issue might have a substantial impact on productivity . Ex: too complex methods, package cycles, etc.
Minor	This issue might have a potential and minor impact on productivity . Ex: naming conventions, Finalizer does nothing but call superclass finalizer, etc.
Info	Unknown or not yet well defined security risk or impact on productivity.

Maintainability

Name	Description
Code Smells	Number of code smells.
New Code Smells	Number of new code smells.
Maintainability Rating (formerly SQALE Rating)	Rating given to your project related to the value of your Technical Debt Ratio. The default Maintainability Rating grid is: A=0-0.05, B=0.06-0.1, C=0.11-0.20, D=0.21-0.5, E=0.51-1 The Maintainability Rating scale can be alternately stated by saying that if the outstanding remediation cost is: • <=5% of the time that has already gone into the application, the rating is A • between 6 to 10% the rating is a B • between 11 to 20% the rating is a C • between 21 to 50% the rating is a D • anything over 50% is an E
Technical Debt	Effort to fix all maintainability issues. The measure is stored in minutes in the DB. An 8-hour day is assumed when values are shown in days.
Technical Debt on new code	Technical Debt of new code

Technical Debt Ratio	Ratio between the cost to develop the software and the cost to fix it. The Technical Debt Ratio formula is: $\text{Remediation cost} / \text{Development cost}$ Which can be restated as: $\text{Remediation cost} / (\text{Cost to develop 1 line of code} * \text{Number of lines of code})$ The value of the cost to develop a line of code is 0.06 days.
Technical Debt Ratio on new code	Ratio between the cost to develop the code changed in the leak period and the cost of the issues linked to it.

CPD Analysis

This automated analysis summarizes the code fragments that are duplicated in custom code. Only fragments longer than 30 lines of code are shown. Here SAP Commerce Cloud will provide a summary of the duplication while the full report will be provided with SAP Commerce Clouds' final review document.

Detailed information regarding duplicated code (packages, classes, etc) can be found in the Sonar Report, attached to this review.

<u>Density</u>	4.6%
<u>Duplicated Lines</u>	2,310
<u>Duplicated Blocks</u>	71
<u>Duplicated Files</u>	43

Duplication of code is inside an acceptable threshold. Nevertheless, we still recommend checking this value often.

Complexity Report

This metric measures the cyclomatic complexity, also known as McCabe metric. Whenever the control flow of a function splits, the complexity counter gets incremented by one. Each function has a minimum complexity of 1. It is recommended to keep a low value to improve readability and maintenance of the code.

<u>Cyclomatic Complexity</u>	3,379
<u>Cognitive Complexity</u>	1,708

Code complexity is inside an acceptable threshold. Nevertheless, we still recommend checking this value often.

Expressions should not be too complex

The complexity of an expression is defined by the number of `&&`, `||` and condition `?` `ifTrue` : `ifFalse` operators it contains.

A single expression's complexity should not become too high to keep the code readable.

```
177
178     protected static OrderEntryData getCartEntry(final CartData cart, final String productCode, final String pickupStore)
179     {
180         for (final OrderEntryData oed : cart.getEntries())
181         {
182             if (oed.getProduct().getCode().equals(productCode))
183             {
184                 if ((pickupStore == null && oed.getDeliveryPointOfService() == null)
185                     || (pickupStore != null
186                         && oed.getDeliveryPointOfService() != null
187                         && pickupStore.equals(oed.getDeliveryPointOfService().getName())))
188                 {
189                     return oed;
190                 }
191             }
192         }
193         return null;
194     }
```

Reduce the number of conditional operators (4) used in the expression (maximum allowed 3). 10 days ago L187

[See Rule](#)

[Code Smell](#) [Major](#) [Open](#) [Not assigned](#) 6min effort [Comment](#) [brain-overload](#)

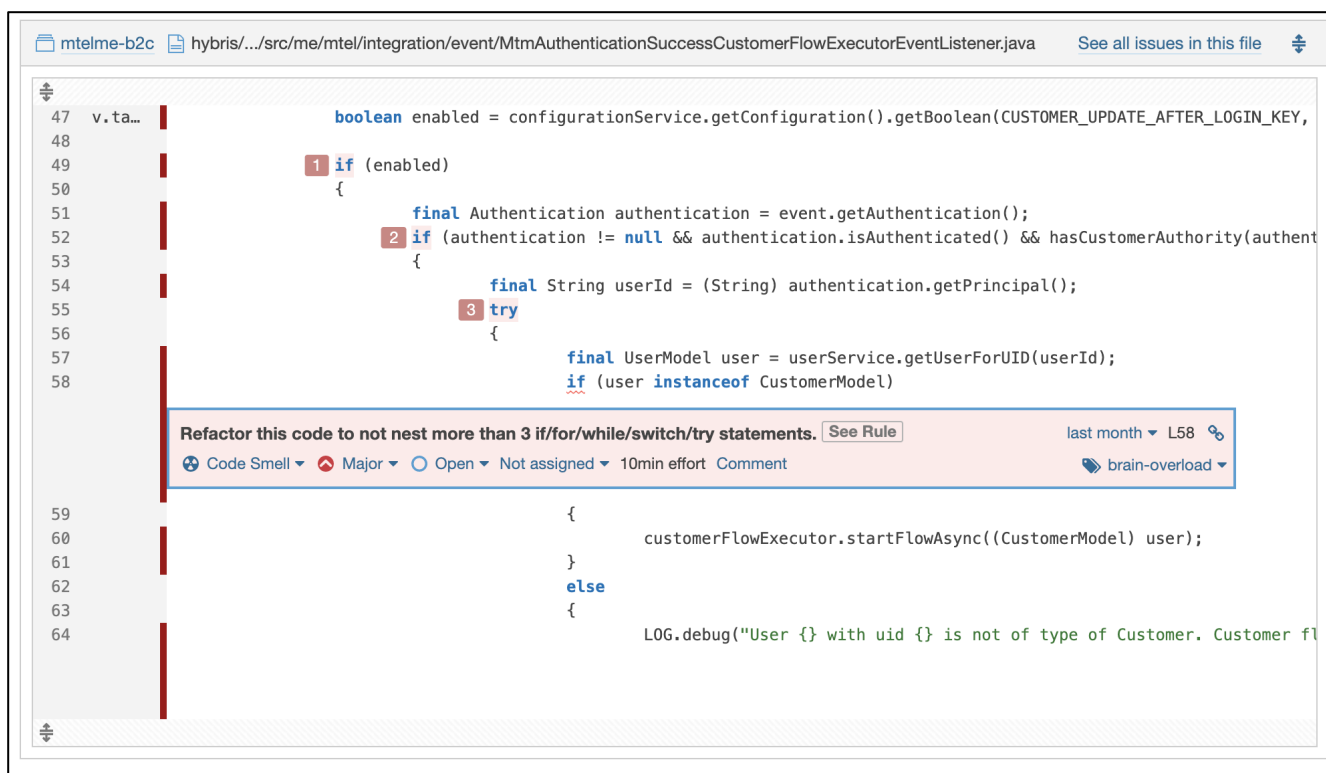
SC-01

Expressions should not be too complex: CartsController:187

Control flow statements "if", "for", "while", "switch" and "try" should not be nested too deeply

Nested if, for, while, switch, and try statements are key ingredients for making what's known as "Spaghetti code".

Such code is hard to read, refactor and therefore maintain.



SC-02

Control flow statements "if", "for", "while", "switch" and "try" should not be nested too deeply: MtmAuthenticationSuccessCustomerFlowExecutorEventListener:58, MtmMobileResourcesInfoPopulator:64

FindBugs and PMD

FindBugs is a program that uses static analysis to look for bugs in Java code; while the PMD tool uses static rules to identify potential problems in the custom code (<http://pmd.sourceforge.net>). The output from these tools was analyzed and we highlight some of the highest priority items.

These contain possible bugs, dead code, empty if/while statements, overcomplicated expressions, suboptimal code, and many other types of potential issues.

Custom code was analyzed both visually and using static-source-code-analysis tools such as PMD. The results of the analysis along with recommendations are provided below.

Issues Type	Number
Blockers	0
Critical	2
Major	94
Minor	307

Exception handlers should preserve the original exceptions

When handling a caught exception, the original exception's message and stack trace should be logged or passed forward.

251
252
253
254
255
256
257
258 a.ma...
259
260 v.ta...
261 a.ma...
262

263
264
265
266 v.ta...
267

```
final List<String> usedCustomerIds = customerModel.getMtmCustomers().stream().map(CustomerModel::getId).collect(Collectors.toList());
final List<String> result = customerIds.stream().filter(id -> !usedCustomerIds.contains(id)).collect(Collectors.toList());

final List<MtmAutoRecognizedServiceData> recognizedServices = new ArrayList<>();
result.forEach(customerId -> {
    final MtmCheckRegisteredServiceData registeredServiceData = reconnectFacade.getMtmCheckRegisteredServiceData(customerId);
    try {
        customerIntegrationService.getRecognizedServices(recognizedServices, customerId);
    } catch (MtmRemoteResponseException e) {
        LOG.error("Mtel system doesn't contain info of {} service", customerId);
    }
});
servicesListData.setRecognizedServices(recognizedServices);
```

SC-03

When handling a caught exception, the original exception's message and stack trace should be logged or passed forward: MtmCustomerFacadeImpl:262,

There are also 94 Major Issues that should be investigated, as they also can affect the system's performance, security and reliability.

Rule	Number of issues
<u>Printf-style format strings should be used correctly</u>	49
<u>"Preconditions" and logging arguments should not require evaluation</u>	21
<u>Standard outputs should not be used directly to log anything</u>	4
<u>Collapsible "if" statements should be merged</u>	3
<u>Track uses of "TODO" tags</u>	3
<u>Utility classes should not have public constructors</u>	3
<u>Control flow statements "if", "for", "while", "switch" and "try" should not be nested too deeply</u>	2

<u>Strings literals should be placed on the left side when checking for equality</u>	2
<u>Try-catch blocks should not be nested</u>	2
<u>Unused "private" methods should be removed</u>	2
<u>"@Override" should be used on overriding and implementing methods</u>	1
<u>Expressions should not be too complex</u>	1
<u>Public types, methods and fields (API) should be documented with Javadoc</u>	1

SC-04

Review and fix Major code issues.

Class Conflicts

There are no class conflicts.

05 DATA MODEL

- [Overview](#)
- [Types](#)
- [Type System Validation](#)
 - [Findings of Type System Validation Rules](#)

Overview

SAP Commerce Cloud Solution ships with a default-data model. However, the product provides mechanisms to extend and adapt the default data-model to customer requirements. Data model adaptations made for the project were reviewed with the Type System Validator.

Types

The SAP Commerce Cloud Solution uses a system of types to organize data, for example product information, customer data, addresses, or orders. This “system of types” is known as application’s type-system or data model. This is defined through **items.xml** files and is used to generate code, create database tables, relationship mapping, and data access with SQL. It is imperative that the type-system is defined according to best practices, as it defines the basic building blocks of the application. Inefficiencies within the type-system definition can permeate through the entire application.

Type System Validation

SAP Commerce Cloud Solution uses a proprietary tool called the TSV (Type System Validator) to ensure items.xml type and attribute definitions are in line with SAP Commerce Cloud Solution best practices. This tool was run over all of mtel.me - B2C’s items.xml files. The output was then analyzed to produce a list of the highest priority items that required changes. The full output will be provided with SAP Commerce Cloud Solution’s final Review Document.

Findings of Type System Validation Rules

Deployment Table Must Exist For Item Extending Generic Item

Deployment tables in SAP Commerce Cloud Solution are used to map the logical data model of an extension to physical database tables. SAP Commerce Cloud Solution recommends specifying a deployment for direct subtypes of GenericItem only. If no deployment table is specified for a subtype of GenericItem, the instances of that subtype are stored in the same table as instances of GenericItem. Storing instances of many different types in one single database table causes that database table to have quite a lot of columns to store all attributes from all these types.

It may have a critical impact on performance in the following cases:

- SQL joins against the generic table.
 - Table scans performed against the generic table.
 - Limitations on query optimizers
 - Record insertion, updates, and deletes may be considerably slower as more indexes and constraints are added to the model.
- Types mapped to the generic table are also limited in the respect that:

- Unique key constraints may be impossible to define since many types share a single table.
- Common attributes must have the same physical constraints for all types in the generic table. For example, an attribute such as code cannot be specified as a maximum length of 10 for type A and a maximum length of 20 for type B.

The above reasoning is also valid for many-to-many relations, and thus they should be defined with their own deployment table.

This review found that the following types and relations should be fixed:

A deployment table must be defined for all Items extending GenericItem.

OK

No Deployment Table Should Exist For Item If Not Extending Generic Item

Specifying deployment tables on sub---types where the super type has a deployment table defined can result in large and complex queries, which can impact performance. Searching a super type will result in a SQL Union queries for each of the sub types.

OK

Avoid using reserved type codes

Type codes in deployment clauses up to 10000 are reserved for types defined within the SAP Commerce product.

Developers creating types for customer implementation projects or reusable assets that are not shipped as part of the product must select a type code greater than 10000 that is not listed in the few exceptional reserved numbers above 10000;

Type codes of the following pattern: 132xx are reserved for commons extension

Type codes of the following pattern: 234xx and 239xx are reserved for print extension

Type codes of the following pattern: 327xx are reserved for processing extension

Type codes of the following pattern: 244xx and 245xx are reserved for legacy xprint extension

Type codes of the following pattern: 100xx are reserved for b2bcommerce extension

See [Specifying a Deployment for Commerce Platform Types](#) for more details. Otherwise, there is a strong possibility that you will cause a conflict with a system type code if you use a value less than 10000. Even if there are no conflicts with the current version of the product and the extensions currently being used, an upgrade to a newer version of SAP Commerce, or the addition of extra extensions from the product, could subsequently cause a type code conflict.

OK

Jalo Class Is Not Allowed When Adding Fields To Existing Class

The jaloclass attribute is not allowed when autocreate='false' and generate='false'.

OK

Mandatory Field Must Have Initial Value

Mandatory fields (where optional='false') must either have initial set to 'true' or a default value defined. Mandatory fields for a type —that is, defined with optional='false'— should be declared initial or a default value should be provided. The following attributes are declared mandatory, but neither are initial nor have a default value.

The following boolean fields must be mandatory (having optional='false'):

OK

Immutable Field Must Have Initial Value

Immutable fields —with write='false'— should be declared also initial due to the fact that the setter method of the attribute is not generated so the attribute needs to be created at initialization time.

The following immutable fields (where write='false') must have initial set to 'true':

OK

Boolean Field Cannot Be Optional

The following Boolean attributes must be defined as mandatory:

OK

Type Name Must Start With Uppercase Letter

The following type names (including EnumTypes and Relations) must start with an uppercase letter:

OK

Field Name Must Start With Lowercase Letter

The following Item attribute names should start with a lowercase letter:

Extension	Attribute
mtmcore-items.xml	Customer.JMBG

DM-01

For coding best practices, attribute names should start with an lowercase letter: Customer.JMBG

Ordering Of Relation Should Be Avoided

Any side of a relation that has cardinality='many' should not have ordered='true' unless absolutely necessary. Ordered relations reduce the system performance, especially during export and catalog synchronization. Therefore, it is a good practice to switch order off whenever it is possible. It is also recommended to define the collection type as "set" whenever possible.

Extension	Attribute
mtmcore-items.xml	Customer2MtmCustomer.mtmCustomers
mtmcore-items.xml	MtmTmaProductSpecCharacteristicGroup2TmaProductSpecCharacteristicRelation.productSpecCharacteristics
mtmcore-items.xml	TmaSubscriptionBase2MtmDeviceInfo.devices

DM-02

Any side of a relation that has cardinality='many' should not have ordered='true' unless absolutely necessary.

Database indexes

No database indexes defined for unique attributes of type. Mostly it makes sense to have it in place as it is used as main WHERE clause in queries and there can speedup fetching entries.

Extension	Attribute
mtmcore-items.xml	MtmRegistration
mtmcore-items.xml	MtmOtpRequest
mtmcore-items.xml	MtmDeviceInfo

DM-03

No database indexes defined for unique attributes of type: MtmRegistration, MtmOtpRequest, MtmDeviceInfo.

Logging to DB

DM-04

Logging integration requests to database can cause database overload which can affect storefront performance. Consider using standard log file which is searchable by Kibana.

06 VISUAL CODE ANALYSIS

- [Overview](#)
- [General Inspection](#)
 - [Web Modules](#)
 - [Usage of instanceof Operator](#)
 - [Abortable Jobs](#)
 - [Jobs and Business Process](#)
- [Accelerator/Commerce Extensions](#)
- [Usage of Logging](#)
 - [Configuration](#)
- [Unit Tests](#)
- [Controllers](#)
- [Facades](#)
- [Services](#)
- [DAOs](#)
- [Populators and Converters](#)
- [Value Providers](#)
- [Interceptors](#)
- [Spring](#)
 - [Recommendations](#)
- [Session Management](#)
- [Usage of Services provided by SAP Commerce Solution](#)
- [Cache](#)

Overview

This section contains a manual analysis of the custom code that was written. Each extension was analyzed along with its relationships to code in other extensions. Additionally the spring files, cache configuration, and other global items that are not extension specific were reviewed here.

General Inspection

Web Modules

Some extensions don't need web module but they have it declared in extensioninfo.xml which makes them accessible online and raises security concerns. For example:

```
<webmodule jspcompile="false" webroot="/mtmmockserver"/>

<webmodule jspcompile="false" webroot="/mtmcommercewebservices"/>
```

Only extensions that need to be accessed from URL have a web module declared in their extensioninfo.xml definition file.

Usage of instanceof Operator

The SAP Commerce Solution extensively uses Interfaces and Abstract Classes, therefore it's common to cast instances of objects into parent (*upcasting*) or child classes (*downcasting*). Due to the more restrictive nature of the latest, plus the high modularity in the SAP Commerce Solution, it's safe to always check for casting validity through the instanceof operator. Not doing so may result in a ClassCastException during runtime, which is highly undesirable.

Furthermore, handling such scenarios with try/catch blocks is not only more expensive performance-wise, but handling code flow using exception is not a good practice.

Here's an example of a possible issue:

Code Block 2 instanceof (NOK 01)

```
// myParentObject being a higher class in the hierarchy. This might lead to a
ClassCastException.

final FSMORERestrictiveObject myObject = (FSMORERestrictiveObject) myParentObject;
```

Code Block 3 instanceof (NOK 02)

```
// myParentObject being a higher class in the hierarchy. This might lead to a
ClassCastException.

// Handling this using a try/catch block is more expensive and bad practice in general.
Avoid it.

try {

    final FSMORERestrictiveObject myOtherObject = (FSMORERestrictiveObject) myOtherObject;

} catch (ClassCastException ccExp) {

    // ...

}
```

Instead, validate this cast using instanceof operator is recommended and more performant. Like so:

Code Block 4 instanceof (OK)

```
if (myObject instanceof ParentObject) {

    final FSMORERestrictiveObject myObject = (FSMORERestrictiveObject) myParentObject;

} else {

    // ...

}
```

The instanceof operator is used to correctly check before downcasting.

Abortable Jobs

Custom Jobs are not abortable by default, which means it won't be possible to stop them if they are running for too long. To enable this feature, override the isAbortable() method in any customized MyJobPerformable class:

```
@Override

public boolean isAbortable() {

    return true;

}
```

Alternatively, while extending the `AbstractJobPerformable` bean, you may override its abortable property value in custom spring configuration e.g.

```
<bean id="myJobPerformable"

    class="de.hybris.cronjobtutorial.MyJobPerformable"

    parent="abstractJobPerformable">

    <property name="l10nService" ref="l10nService" />

    <property name="abortable" value="true"/>

</bean>
```

Currently the job is marked as abortable, which allows to activate the abort flag for the cron job, similarly to the **interrupt** flag for **Thread**. When implementing an abortable job it is important to regularly check if the user has requested to abort the job. If true, the abort flag is set, the job should be stopped, and the cron job should have a proper status and result. You should decide when to check the flag and what to do if the flag is set, for example, to clean up. Time of processing between verifications should be moderate.

The following code sample calls the **clearAbortRequestedIfNeeded** of the **AbstractJobPerformable**. It checks if the given **cronJob** is requested to be aborted and has **REQUESTABORT** flag set to **true**. If so, the flag is set to **false** again and the job execution is prematurely stopped.

Code Block 6 abortable job

```
if(clearAbortRequestedIfNeeded(cronJob))  
{  
    //abort the job  
    //do some clean-up  
    return new PerformResult(CronJobResult.ERROR, CronJobStatus.ABORTED);  
}
```

Custom Cron Jobs:

Cron Job name	Abortable	Comment
CustomerFlowJobPerformable		CronJob triggers Spring Integration gateway, therefore no need to implement abortable interface.

Cron jobs follow SAP Commerce Solution best practices.

Jobs and Business Process

Custom Jobs have been assessed to verify if they are responsible for directly executing business processes, with the potential load and stability issues that this may cause, as per the example below.

```
modelService.save(newCustomProcessModel);  
getBusinessProcessService().startProcess(newCustomProcessModel);
```

One custom jobs directly creating business processes have been found.

Custom Job

MtmConfirmationEmailEventListener

Accelerator/Commerce Extensions

Spring configuration of main accelerator/commerce extensions should never be directly modified. For that, SAP Commerce Solution provide aliases to internal Spring beans, which can be used in custom Spring configuration files to override default behaviour.

Core Spring configuration files have not been directly modified. Instead, aliases were used to customize Beans.

Usage of Logging

Configuration

1) Log4j 2 or org.slf4j.Logger are the recommended logging libraries

The use of only one logging library is consistent with SAP Commerce Solution best practices.

2) The controllers, facades, services, daos, interceptors, populators... as follows do not declare any logger:

Controllers, facades, services, daos, interceptors and populators have declared a logger for recording warnings and errors.

3) The classes as follows use `System.out.println`:

No classes use `System.out.println` to log messages.

Unit Tests

Unit test should be compliant with the following rules:

- Tests must be Fast, Independant, Repeatable, Self-Validating, Timely (F.I.R.S.T)
- Assertion should make sense
- Integration tests should only be implemented where Unit tests are insufficient
- **@Before** should be used appropriately to perform pre-requisite work to be executed before each test.
- Integration tests must have their own test data, don't use actual project data
- Test data must be relevant, i.e. correspond to real-world cases.

JUnit is the industry standard automated testing framework for Java applications. Automated testing is crucial to the development of quality software; poor test coverage is an indicator of poor development practices.

The test-coverage results as follows have been calculated by *jacoco* (Java Code Coverage) during execution of junit tests:

Coverage 30.4%			
	Coverage	Uncovered Lines	Uncovered Conditions
mtmbackoffice	0.0%	45	14
mtmcommerceservices	4.2%	4,135	37
mtmcore	53.7%	833	201
mtmfacades	56.4%	471	86
mtminitialdata	6.6%	130	26
mtmintegration	69.3%	271	167
mtmmockserver	0.0%	228	–
mtmpatches	63.0%	16	1
mtmtest	7.4%	327	86
9 of 9 shown			

JUnit tests have been mocked with Mockito framework, which avoid database access and ensure tests are repeatable. There is adequate test coverage and all tests pass.

Controllers

The project avoids controller-driven development according to SAP Commerce Solution best practices.

Facades

Facades are constructed correctly and are not aware of the DAO layer. DTO are converted using converters outside of the facade layer.

Services

Services use DAOs correctly. No service-layer-flexible-search query has been identified.

DAOs

The code base has no instances of hard-coded-flexible-search queries.

Populators and Converters

Populators and converters within SAP Commerce Solution can be critical for performance considerations. One common mistake when working with converters and populators is to confuse them. Converters should use populators and not vice versa. Wiring in converters into populators goes against their fine-grained nature and can lead to a long conversion hierarchy of unnecessary object creation and garbage collection.

There are no heavy populator implementations dependent on converters.

Value Providers

Value providers can handle the conversion between SAP SAP Commerce Solution database entries and Solr document values. It's their responsibility to retrieve information in order to de-normalize/complete data for an Indexed Type or its properties. There are multiple value providers already available for the most common types that could be used, and custom ones can be created if needed. Keep in mind that Value Providers are used extensively during indexing operations, and for that, they should be carefully designed and unit tested, in order to avoid issues related to performance and behavior.

The use of value providers, especially custom ones, should be used sparingly as they are used multiple times during indexing operations. Non-performant providers can drastically increase the time needed to complete a full index job. Carefully design queries using Flexible Search and minimize loops. Also, consider adopting strategies to improve performance such as Caching and Database Indexes.

Consider if a custom Value Provider is really required. Some requirements can be achieved by using [SpEL](#) Value Providers

Additional guidelines for Value Providers:

1. There should not be multiple DB calls (as DB IO would stack over multiple calls/multiple DBs);
2. There should not be recursive calls (again a Value Provider is called many times for each registered attribute, from each document to be indexed, recursiveness would add to that call stack costing performance);
3. There should not be any unused value provider (they would be called for no good reason, only adding to the processing time of each document).

Value Providers are properly written and no unused Value Provider was found.

Interceptors

The goal of the interceptor should be to ensure data integrity (interceptors are low level), it should not execute complex business logic as it is better to keep the logic in a proper service. Interceptors can have a large influence on the performance on a system if not implemented correctly.

Interceptors :

- Do not contain any complex business logic.
- Are well-written so that in principle they could not produce an overhead when initializing, saving or removing a model.

Spring

Way to get a spring bean from application context

Spring beans are correctly retrieved from application context, using :

- bean name only
- alternative : bean name and bean class.

example : `Registry.getApplicationContext().getBean("configurationService", ConfigurationService.class)`

Spring inheritance should be leveraged in custom extensions for maintainability and upgradability.

Recommendations

Spring inheritance has been leveraged well for beans with overlapping properties.

Use dependency injection when possible instead of using `SpringUtil.getBean()`.

No instances of `SpringUtil.getBean()` have been found.

Some custom bean names start with uppercase. The Spring bean naming convention is to use the standard Java convention for instance field names when naming beans. That is, bean names start with a lowercase letter, and are camel-cased from then on. Overrides bean by name is very common in SAP Commerce Solution, therefore naming beans consistently is important.

All bean names appear to start with lowercase.

If alias is defined, make the injected property to reference to alias instead of specific bean name. In this way, the effect of changing alias will be propagated to all beans referencing it. Also, creating an alias with coinciding name and alias properties doesn't have any effect. Other common issues we look for are bean names and Java classes prefixed with "default" as this should only be used for a specific bean instance. Even if it's a default implementation, it's better to prefix by the project name, that is `EasyProductService.java` instead of `DefaultProductService.java`.

Beans are appropriately referenced by alias as opposed to bean name.

Overall, the whole project does not have a consistent way to do dependency injection. @Resource, @Autowired and xml configuration are used.

The project is consistent in its use of dependency injection.

Session Management

Sessions within an application should not store excessively large objects or models. Also it is important to check that all such values stored in the sessions are relevant and not being used by calls outside of the session. This can cause severe degradation of performance caused by memory saturation.

HTTP and Jalo sessions do not seem to contain large objects or Models.

Usage of Services provided by SAP Commerce Solution

Check if `de.hybris.platform.servicelayer.config.ConfigurationService` is used.

The project code uses `de.hybris.platform.servicelayer.config.ConfigurationService` rather than `de.hybris.platform.util.Config`.

Check if `de.hybris.platform.servicelayer.search.FlexibleSearchService` is instead of `FlexibleSearch`

`FlexibleSearchService` is used.

Check if `de.hybris.platform.servicelayer.user.UserService` is used to get the current user or a user by UID.

The JALO layer has been deprecated and should be avoided wherever it is feasible. SAP Commerce Solution recommends using `UserService` instead. Similarly, **`UserService`** should be used to get User by UID instead of **`UserManager`**.

`UserService` is used to get the current user, which is recommended.

Cache

There are areas of high utilization where performance will be critical and potential optimizations of these areas of code could be to introduce caches. One area where caches are most utilized is around Converter and Populator classes, so that repeated conversions are not necessary. SAP Commerce Solution also provides a package in this area which adds caching layers into the storefront project for the most commonly seen performance considerations.

Also, for a more detailed cache investigation, please consider a [System Assessment](#). Furthermore, if you want to modify cache to adhere to performance requirements, a [Performance Assessment](#) is recommended.

VCA-01

Application uses default API and region cache configuration. Fine tune cache configuration based on the performance tests executed before Go-Live.

Other checks

VCA-02

Implement configurable timeouts for integration interfaces.

VCA-03

Remove V1 version REST API if not used.

07 BACKOFFICE CONFIGURATION

Overview

The SAP Commerce Cloud Solution Backoffice/internal applications need to be configured and sometimes extended to meet the requirements of the customer. Some of the key back office/internal tools that might need customization were reviewed and below are the findings. Also discussed in this section, which is global to all back office/internal tools and the storefront, is localization.

At the very least these tools should be configured to handle company-specific-data-model types and have localized labels. This analysis did not take into account which back office tools will be used or whether the back office tools configuration incorporates the functionality that is required by the business. This analysis identifies which tools have been configured, if project-specific items still require customization, and if there are specific issues with the way the custom configuration has been implemented.

Deprecated Features

HMC was de-released in release 6.7. Additionally the cockpit framework has been deprecated (including product cockpit) and is set to be de-released by end of 2018.

extensioninfo.xml does not contain the hmc extension or extensions related to the cockpit framework

No customizations were done in hmc or cockpit framework related extensions

Backoffice - Application Configuration

The Backoffice can be customized with custom Backoffice modules, Widgets or UI configuration. All Backoffice customizations must be saved in custom extensions that extend ybackoffice extension template. Each extension implemented using ybackoffice will contribute to the main Backoffice application. The recommended approach is to design the widget mashup using Application Orchestrator and copy the generated `${HYBRIS_DATA_DIR}/backoffice/config/backoffice-widgets.xml` to your custom extension, so that the configuration will be persisted.

Performing a reset in Application Orchestrator will result in the configuration from all backoffice module extensions to be loaded.

Backoffice customizations are implemented in custom backoffice extensions

Custom Backoffice Behavior

Backoffice is highly extensible and configurable. Application mashups can be designed and built to manage any kind of data. You can build widgets and combine and connect widgets to create a backoffice business process quickly. Additionally you can create perspectives.

Custom backoffice behaviour works as expected.

Backoffice Folder Structure

The backoffice framework defines a directory structure for widgets, actions and editors to manage its own configuration files.

The following files are relevant within a custom backoffice extension, located under the **resources** folder:

-----<custom backoffice extension>/resources/

-----<custom backoffice extension>-backoffice-widgets.xml: holds the default configuration of components defined in the custom backoffice extension

-----<custom backoffice extension>-backoffice-config.xml: holds configuration of components defined in the custom backoffice extension.

-----<custom backoffice extension>-backoffice-labels/labels*.properties (optional) :holds localized labels

The following files are relevant within a custom backoffice extension, located under the **backoffice** folder:

-----<custombackoffice extension>/backoffice/resoures/widgets/

-----<widget>/definition.xml: holds the definition of a specific widget. The component definition, inputs and outputs, and all meta data related to it.

-----actions/<action>/

-----definition.xml: holds action definition

-----labels/labels*.properties (optional): holds localized labels for action

-----editors/<editor>/

-----definition.xml: holds action definition

-----labels/labels*.properties (optional): holds localized labels for action

Backoffice directory structure and configuration files for widgets, action and editors meet recommended guidelines

Localized labels are defined for widgets, actions and editors

Customer Specific Data Model

Backoffice needs to handle the customer specific data model. Project specific custom types need to be represented in backoffice. Review the types within backoffice and ensure that company specific data model is represented.

Company specific data model is represented

Localization

Localization files contain the language translations for the attributes and types defined in the items.xml files. To ensure the backoffice tools have meaningful labels in the language of the business users it is important that all custom types and attributes have been represented on the localization files. Even if the only language that the backoffice tools are used in is English, these descriptive names should still be present.

All types and attributes have been represented in localization files.

www.sap.com/contactsap

© 2019 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

The information contained herein may be changed without prior notice. Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors. National product specifications may vary.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP or SAP affiliate company products and services are those that are set forth in the express warranty statements.

SAP Customer Experience

THE BEST RUN

